杭州电子科技大学

计算机学院

编译原理课程实践

# 项目 1　预备实验

从源代码到可执行文件：GCC 工具链观察与 Flex 词法分析入门

实　验　报　告

| | |
|---|---|
| 课程名称 | **编译原理课程实践** |
| 课程代码 | **S0512080** |
| 专业班级 | |
| 学　　号 | |
| 姓　　名 | |
| 指导教师 | |
| 完成日期 | **2026 年 10 月 31 日** |

# 目 录

目 录 2
第 1 章 环境配置 3
1.1 平台与工具版本 3
1.2 环境自检 3
第 2 章 走近 GCC：从源代码到可执行文件 4
2.1 GCC 是驱动程序 4
2.2 编译、运行与产物类型 4
2.3 读一条错误信息 4
2.4 多文件与链接错误 5
第 3 章 阶段产物观察 6
3.1 四阶段与五种文件 6
3.2 预处理：宏消失，头文件铺开 6
3.3 编译与汇编：从 C 文本到目标文件 7
3.4 链接：符号解析与重定位 7
3.5 -### ：只打印计划，不执行 8
第 4 章 编译选项实验：性能与诊断的取舍 9
4.1 实验 1：优化等级对比 9
4.2 实验 2：警告与错误 10
4.3 实验 3：调试信息 11
4.4 实验 4：预处理宏 11
4.5 结论：按场景选选项 12
第 5 章 Lex 工具入门：编写与测试简单的词法分析器 13
5.1 三段式结构与工作流 13
5.2 实验 1：最简整数识别器 13
5.3 实验 2：多类数字字面量 13
5.4 实验 3：关键字、标识符与整数 15
5.5 最长匹配与等长规则优先级 16
5.6 从正规式到自动机 16
第 6 章 思政融合：国产编译器调研 17
6.1 调研对象 17
6.2 关键发现 17
6.3 我的思考 17
6.4 参考文献 18
第 7 章 综合反思 19
7.1 最有趣的环节 19
7.2 最大收获 19
7.3 还存在的疑问 19
附录 A 提交文件清单与自行完成部分 20
A.1 numbers.l （自行完成部分） 20
A.2 mini_c.l （自行完成部分） 20

# 第 1 章　环境配置

## 1.1 平台与工具版本

表 1-1　实验环境（版本号均由 `--version` 直接读出）

| 项目 | 实测值 |
| --- | --- |
| 操作系统 / 架构 | Windows 11（10.0.26200.9711），x86_64 |
| 终端 | MSYS2 / MinGW64，GNU bash 5.3.15 |
| C 编译器 | GCC 15.2.0（x86_64-win32-seh-rev0，MinGW-Builds），目标 `x86_64-w64-mingw32` |
| 汇编器 / 链接器 | GNU as / GNU ld（Binutils 2.45） |
| 目标文件格式 | COFF；可执行文件 PE32+ |
| 调试器 / 词法工具 | GNU gdb 16.3；win_flex 2.6.4 |
| 观察工具 | nm、objdump、size、file、ldd |

任务书首选 Ubuntu/WSL2，本机 WSL 不可用，故改用 MinGW-w64 原生工具链。四阶段的划分、符号解析与重定位机制与 Linux 完全一致，差异集中在三处，后续章节遇到时会结合实测说明：目标文件是 COFF 而非 ELF（`file` 报 `PE32+`）；可执行文件带 `.exe` 后缀；缺少 `main` 时链接器报 `undefined reference to 'WinMain'` 而非 `'main'`——因为 MinGW 的 C 运行时启动对象同时接受 `main` 与 `WinMain` 两个入口名，规则相同、实现不同。

## 1.2 环境自检

**终端实录 1-1**　编译运行、符号检查与动态依赖

```
$ gcc -Wall -Wextra -O0 -g check.c -o check.exe
$ ./check.exe
add(2,3)=5
$ nm check.exe | grep -E " (T|t) (main|add)$"
0000000140001440 T add
0000000140001454 T main
$ ldd check.exe
        ntdll.dll => /c/WINDOWS/SYSTEM32/ntdll.dll
        KERNEL32.DLL => /c/WINDOWS/System32/KERNEL32.DLL
        ucrtbase.dll => /c/WINDOWS/System32/ucrtbase.dll
```

`-Wall -Wextra` 下无警告；`T` 表示符号位于代码段且已定义，`add` 与 `main` 地址差 `0x14` 字节，说明 `-O0` 下函数边界保留、未被内联；动态依赖只有系统 DLL 与 UCRT。随后 GDB 能加载带调试信息的程序并正常退出，`win_flex` 生成的识别器对输入 `123 hello 456` 输出 `INT: 123` 与 `INT: 456`，自检六项全部通过。

# 第 2 章　走近 GCC：从源代码到可执行文件

## 2.1 GCC 是驱动程序

`gcc` 本身几乎不做编译工作，它是驱动程序：按参数决定停在哪个阶段，再依次调起 `cc1`（C 编译器）、`as`（汇编器）、`collect2` / `ld`（链接器）。因此 `-E` / `-S` / `-c` 的含义是"停在哪个阶段之后"，而 `-o` 只决定输出文件名，与停止位置无关。

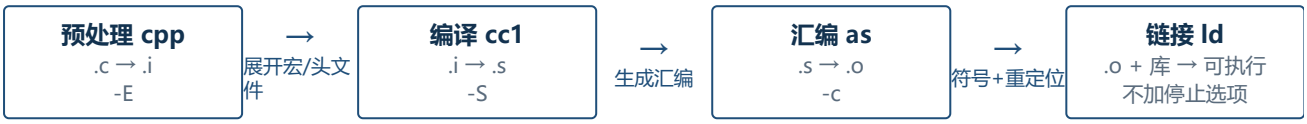



**图 2-1**　GCC 工具链四阶段与停止选项

## 2.2 编译、运行与产物类型

源文件 `hello.c` 含宏 `GREETING`、函数 `add` 与一次格式化输出（完整源码随 `src/` 提交）。

**终端实录 2-1**　最简编译流程

```
$ gcc hello.c -o hello.exe
$ ./hello.exe
Hello, Compiler! 2 + 3 = 5
$ file hello.exe
hello.exe: PE32+ executable for MS Windows 5.02 (console), x86-64, 16 sections
```

`file` 只报告容器格式（PE32+）、平台（MS Windows 控制台）、指令集（x86-64）与节数（16），语言的痕迹已经完全消失——从 `.c` 到可执行文件，剩下的只是操作系统能装载的二进制结构。

## 2.3 读一条错误信息

`bug.c` 在 `printf("hello"` 后缺少右括号与分号：

**终端实录 2-2**　语法错误诊断

```
$ gcc bug.c -o bug.exe
bug.c: In function 'main':
bug.c:3:19: error: expected ')' before 'return'
    3 |     printf("hello" // 缺少右括号
      |            ~       ^
      |                    )
    4 |     return 0;
      |     ~~~~~~
bug.c:4:14: error: expected ';' before '}' token
    4 |     return 0;
      |              ^
      |              ;
```

诊断分四层：**定位**（`3:19` 即字符串字面量结束处）、**性质**（`error` 而非 `warning`，不会生成目标文件）、**期望**（此处期待函数调用的收尾却读到 `return`）、**建议**（用 `^` 指位并给出补上 `)` 的修复示意）。第二条 `expected ';'` 是级联错误：解析器按"已补上右括号"的假设继续读，于是又发现 `return 0;` 缺分号。所以错误要自上而下逐条修，改一条重编译一次。`bug_fixed.c` 补上 `)` 与 `;`（另加 `\n` 防止输出与提示符同行）后编译运行正常。

## 2.4 多文件与链接错误

**终端实录 2-3** 缺少定义时的链接错误，以及补齐后的成功链接

```
$ gcc main.c -o broken.exe
ld.exe: .../Temp/ccjDSCxr.o:main.c:(.text+0x18): undefined reference to `add'
collect2.exe: error: ld returned 1 exit status
$ gcc main.c add.c -o two_files.exe
$ ./two_files.exe
3 + 5 = 8
```

报错里的三个细节对应链接器的职责：`ccjDSCxr.o` 说明编译已成功、失败发生在链接阶段；`(.text+0x18)` 定位到 `call add` 那条指令，链接器知道"这里要填地址但填不上"；`undefined reference` 表示符号解析失败——所有输入目标文件与库里都没有名为 `add` 的已定义符号。而编译 `main.c` 时不报错，是因为 C 采用分别编译：编译器只核对 `extern int add(int, int);` 声明与调用是否匹配，"定义是否真的存在"由链接器负责。

表 2-1 任务 1.1 三个验收点

| 序号 | 验收点 | 证据 |
| --- | --- | --- |
| ① | HelloWorld 正确输出 | 实录 2-1，输出与预期逐字符一致 |
| ② | 能读懂 GCC 错误信息 | 2.3 节：定位、期望、修复提示、级联错误逐个解释 |
| ③ | 能正确编译多文件项目 | 实录 2-3：先复现链接错误，再输出 `3 + 5 = 8` |

# 第 3 章　阶段产物观察

## 3.1 四阶段与五种文件

工具链有四个阶段，磁盘上出现五种文件——多出来的那一种是 `.c`，它是整条流水线的输入，不属于任何阶段的输出，所以"五种文件"不等于"五个阶段"。

表 3-1　四阶段工具链与实测产物（对 `hello.c`）

| 阶段 | 工具 | 输入 | 输出 | 选项 | 本机实测 |
|---|---|---|---|---|---|
| 预处理 | `cpp` | `.c` + 头文件 | `.i` | `-E` | 1526 行（78 KB），98 个 `#` 行标记 |
| 编译 | `cc1` | `.i` | `.s` | `-S` | 57 行（1125 B） |
| 汇编 | `as` | `.s` | `.o` | `-c` | COFF 1106 B，7 节 22 符号，代码段 112 B |
| 链接 | `ld` | `.o` + 库 | 可执行 | 不加停止选项 | 53 574 B 的 PE32+ |

## 3.2 预处理：宏消失，头文件铺开

**终端实录 3-1**　生成并检查 `.i`

```
$ gcc -E hello.c -o hello.i
$ wc -l hello.i
1526 hello.i
$ grep -c '^#' hello.i
98
$ grep -n "Hello, Compiler" hello.i
1524:    printf("%s %d + %d = %d\n", "Hello, Compiler!", x, y, add(x, y));
```

三个结论：宏已不存在——源文件里的 `GREETING` 在第 1524 行只剩 `"Hello, Compiler!"` 字面量，预处理是纯文本替换；1526 行里绝大部分是 `<stdio.h>` 递归包含的内部头文件，自己写的只有 8 行；以 `#` 开头的 98 行不是未展开的宏，而是行标记（`# 行号 "文件名" 标志`），供编译器报错时反查源码位置。用 `-P` 抑制行标记后行数降为 1165，差额 361 行即标记与空行。

## 3.3 编译与汇编：从 C 文本到目标文件

**终端实录 3-2** 生成 `.s` 与 `.o` 并观察

```
$ gcc -S hello.i -o hello.s
$ wc -l hello.s
57 hello.s
$ grep -n -E '^_?main:' hello.s
29:main:
$ gcc -c hello.s -o hello.o
$ file hello.o
hello.o: x86-64 COFF object file, no line number info, not stripped,
         7 sections, symbol offset=0x2ac, 22 symbols, ...
$ nm hello.o
0000000000000000 T add
0000000000000014 T main
                 U printf
                 U __main
$ objdump -r hello.o
RELOCATION RECORDS FOR [.text]:
OFFSET           TYPE                     VALUE
000000000000001d IMAGE_REL_AMD64_REL32    __main
0000000000000047 IMAGE_REL_AMD64_REL32    .rdata
000000000000004e IMAGE_REL_AMD64_REL32    .rdata
0000000000000057 IMAGE_REL_AMD64_REL32    printf
```

`.s` 里 `add:` 在第 6 行、`main:` 在第 29 行，与源码顺序一致；本机汇编带大量 `.seh_*` 伪指令，是给链接器提供 Windows x64 结构化异常处理所需的栈展开信息，Linux 上没有。目标文件是信息量最大的一环：`T` 表示已定义符号（位于 `.text` 段），`U` 表示"本单元用到但没定义"，要留给链接阶段在 C 运行库里解决；`.rdata` 存字符串常量；4 条 `REL32` 重定位记录说明这些指令里的地址暂时填 0，等链接器确定最终地址后回填。`__main` 是 MinGW 特有的运行库初始化调用，Linux 的 GCC 不产生——这正是"可重定位目标文件必须留未定义符号"的原因：**编译一个文件时，编译器不可能知道别的文件或运行库里有什么**。

## 3.4 链接：符号解析与重定位

**终端实录 3-3** 单独链接 `hello.o` 即可运行；两个 `.o` 组合同样可行

```
$ gcc hello.o -o hello_from_o.exe && ./hello_from_o.exe
Hello, Compiler! 2 + 3 = 5
$ gcc main.o add.o -o two_files_o.exe && ./two_files_o.exe
3 + 5 = 8
$ gcc add.o -o no_main.exe
ld.exe: .../libmingw32.a(...crtexewin.o):crtexewin.c:(.text.startup+0xb4):
        undefined reference to `WinMain'
collect2.exe: error: ld returned 1 exit status
```

只给 `.o` 就能链接成功，说明链接器需要的是符号表与重定位表，不需要源码：`printf`、`__main` 由 C 运行库提供（`ldd` 显示依赖 ucrtbase 等），4 个待填地址在链接时回填。缺少入口的报错与指南的 Linux 版本（`undefined reference to 'main'`）文本不同、原因相同：链接器把 C 运行时启动对象也链了进来，它需要一个入口符号交出控制权，MinGW 实现里最后尝试的是 `WinMain`。

## 3.5 `-###`：只打印计划，不执行

**终端实录 3-4** `-###` 打印的流水线（节选）

```
$ gcc -### hello.c -o hello_trace 2> trace.txt
$ wc -l trace.txt
17 trace.txt
$ grep -o "cc1\.exe\|as\.exe\|collect2\.exe\|ld\.exe" trace.txt | sort | uniq -c
      1 as.exe
      1 cc1.exe
      1 collect2.exe
      1 ld.exe
$ ls hello_trace*
未生成（符合预期）
```

**打印出的命令证实了三点：四阶段是被"接力"执行的——`cc1` 的输出直接写成临时 `.s`，`as` 拿它写成临时 `.o`，再交给 `collect2`（`ld` 的包装器，额外负责收集构造/析构信息），临时文件名每次随机生成；`-###` 确实什么都没生成，与 `-v`（真的编译并打印）不同；所谓"驱动程序"，就是把这几条命令按依赖顺序拼起来执行的那个角色。**

# 第 4 章　编译选项实验：性能与诊断的取舍

## 4.1 实验 1：优化等级对比

被测程序是求和循环 `sum = Σ(2i+1) = N²`（`N = 10 000 000`，结果应为 `100000000000000`，四个版本输出均正确）。`clock()` 只包住循环区间，排除 `printf` 影响；每个版本采样 9 次取中位数。

表 4-1　优化等级比较记录表（本机实测）

| 选项 | 文件大小 / B | 代码段 / B | 汇编行数 | 循环 CPU 时间 / 秒（9 次采样） | 中位数 |
|---|---|---|---|---|---|
| -O0 | 54 517 | 11 420 | 89 | 0.013 0.009 0.008 0.010 0.009 0.017 0.011 0.013 0.015 | 0.011 |
| -O1 | 54 517 | 11 324 | 72 | 0.003 0.003 0.004 0.007 0.003 0.003 0.005 0.005 0.003 | 0.003 |
| -O2 | 54 553 | 11 324 | 68 | 0.000000 ×9（全部为 0） | 无法可靠区分 |
| -O3 | 54 553 | 11 324 | 68 | 0.000000 ×9（全部为 0） | 无法可靠区分 |

**最意外的发现**：`-O1` 的汇编里已经出现 `movabsq $100000000000000, %rdx`——`sum` 被常量折叠了，但循环本身被降级成空转计数器留了下来：

```
# -O1 生成的 main 片段
        call    clock
        movl    $10000000, %eax
        .p2align 3
.L2:
        subq    $1, %rax
        jne     .L2
        call    clock
```

而 `-O2` / `-O3` 连空循环也删了，两次 `clock()` 紧挨在一起，测量值落到计时分辨率以下，故按指南要求标注无法可靠区分（也不能为了让计时"有数"而加 `volatile`，那会改变本实验要观察的优化条件）。其余三点补充：文件大小不能用来看优化效果——PE 节对齐把几十字节的差异掩盖了，改用 `size` 读代码段才看得出 `-O0 → -O1` 的一次性收益（11 420 → 11 324，之后不再变化）；用 `gcc -Q --help=optimizers` 统计各档位打开的优化器数量为 51 / 93 / 142 / 155，说明 `-O1 → -O2` 增加的 49 个开关里大部分与向量化前置分析有关；终端 `time` 两个版本 `real` 几乎相同（0.68 s / 0.69 s），几乎全花在进程创建上，与 11 ms 的循环无关——两种计时测的根本不是同一段时间。

## 4.2 实验 2：警告与错误

表 4-2 `-O1 -Wall -Wextra` 报出的 5 类诊断与修复

| # | 类别 | 位置 | 缺陷本质 | 修复 |
|---|---|---|---|---|
| 1 | `-Wreturn-type` | 2:28 | 非 void 函数执行流可能走到 `}`，返回值不确定 | 补 `return 0;` |
| 2 | `-Wunused-variable` | 4:9 | 声明后从未使用，通常是重构残留 | 删除 |
| 3 | `-Wparentheses` | 7:9 | `if (y = 5)` 是赋值不是判等，条件恒真 | 改 `y == 5` |
| 4 | `-Wformat=` | 8:14 | `%d` 配 `char *`，64 位平台只取指针低 32 位，输出垃圾值 | 改用 `%s` 或 `%d` 配整数 |
| 5 | `-Wuninitialized` | 9:12 | 读取未初始化变量，值不确定（未定义行为） | 初始化 `int x = 0;` |

**终端实录 4-1** `-Werror` 把同一批警告升级为错误

```
$ gcc -O1 -Wall -Wextra -Werror -c warn.c -o warn.o
warn.c:7:9:  error: ... [-Werror=parentheses]
warn.c:8:14: error: ... [-Werror=format=]
warn.c:4:9:  error: ... [-Werror=unused-variable]
warn.c:2:28: error: ... [-Werror=return-type]
warn.c:9:12: error: ... [-Werror=uninitialized]
$ ls warn.o
warn.o 未生成（-Werror 下编译失败，符合预期）
```

消息里选项名从 `-Wparentheses` 变成 `-Werror=parentheses`，标明"是谁把它变成错误的"。其中 3、4、5 条单独看都"能编译能运行"，但都是未定义行为，不会崩溃只会输出垃圾值，必须靠警告发现——这也是指南注明"本例只用于诊断，禁止链接运行"的原因。修复版 `warn_fixed.c` 在同样选项下零警告、编译通过。工程做法：开发期用 `-Wall -Wextra` 暴露问题，CI 期加 `-Werror` 防退化。

## 4.3 实验 3：调试信息

**终端实录 4-2** 同一组 GDB 命令在带 / 不带 `-g` 下的对比

```
$ gcc -O0 -g test.c -o test_with_gdb.exe
$ gdb --batch -x gdbcmds.txt ./test_with_gdb.exe
Breakpoint 1 at 0x14000144d: file test.c, line 3.
Thread 1 hit Breakpoint 1, main () at test.c:3
3           int x = 40;
4           x += 2;
8           printf("answer = %d\n", x);
$1 = 42
x = 42

$ gcc -O0 test.c -o test_no_gdb.exe
$ gdb --batch -x gdbcmds2.txt ./test_no_gdb.exe
Breakpoint 1 at 0x14000144d
Thread 1 hit Breakpoint 1, 0x00007ff60851144d in main ()
Single stepping until exit from function main,
which has no line number information.
...
No symbol "x" in current context.
```

三处差异对应 `-g` 加的三种信息，可用 `objdump -h` 验证：带 `-g` 的版本多出 `.debug_line` / `.debug_info` / `.debug_abbrev` / `.debug_aranges` 等 7 个 `.debug_*` 节，不带的为 0。行号映射让断点停在 `test.c:3` 并打印源码行；变量描述让 `print x` 得到 42；没有行号信息时 `next` 退化为"执行到函数返回"，直接跑穿整个 `main`。无 `-g` 不等于完全不能调试——还能下断点、反汇编、看寄存器，只是看不到源码级信息。

## 4.4 实验 4：预处理宏

**终端实录 4-3** `-DDEBUG` 控制条件编译

```
$ gcc -O0 test.c -o test_off.exe && ./test_off.exe
answer = 42
$ gcc -O0 -DDEBUG test.c -o test_on.exe && ./test_on.exe
[DEBUG] x = 42
answer = 42
$ strings test_off.exe | grep -c "\[DEBUG\]"
0
$ strings test_on.exe | grep -c "\[DEBUG\]"
1
```

`strings` 给了硬证据：不定义 `DEBUG` 时调试字符串根本不存在于可执行文件里，不是"运行时被跳过"而是"编译时就没生成"，零开销。`-DDEBUG` 与 `-g` 完全不等价：前者是预处理阶段的文本开关，决定哪些代码进入编译；后者只附加元数据，不改变一行机器码。可以同时用，也可以只用其一。

## 4.5 结论：按场景选选项

表 4-3　编译选项选择依据（依据本实验数据）

| 场景 | 推荐组合 | 依据 |
| --- | --- | --- |
| 调试、单步定位 | `-O0 -g -Wall -Wextra` | `-O0` 保持源码与指令对应，`-g` 提供行号与变量；`-O1` 起局部变量可能被优化掉 |
| 日常开发构建 | `-O1 -Wall -Wextra` | 本实验中 `-O1` 已拿到代码段缩减的主要收益，与 `-O2` 同为 11 324 B |
| 发布 / 性能敏感 | `-O2`（有向量化空间再 `-O3`） | `-O2` 已消除死循环；`-O3` 多出的 13 个开关主要服务向量化 |
| CI 门禁 | `-Wall -Wextra -Werror` | 把 5 类警告变成硬失败，防止未定义行为代码合入 |

一句话：**优化等级越高，机器码越"不像"源码，可诊断性越差；调试信息与警告不改变机器码，却决定问题能不能被发现**。

# 第 5 章　Lex 工具入门：编写与测试简单的词法分析器

## 5.1 三段式结构与工作流

`.l` 文件由 `%%` 分成定义段（选项、C 声明）、规则段（正规式 + 动作）、用户代码段（原样拷入生成文件）。Flex 读它生成 `lex.yy.c`（含 `yylex()`、`yytext`、自动机状态表），再编译成可执行识别器。三者的职责：`.l` 是规则描述，`lex.yy.c` 是规则的实现，可执行文件才是识别器；改规则只改 `.l`，直接改 `lex.yy.c` 会在下次生成时被覆盖。




**图 5-1**　`.l`、生成的 C 文件与可执行文件的关系

## 5.2 实验 1：最简整数识别器

**终端实录 5-1**　生成、编译、运行

```
$ flex demo.l
$ ls -l lex.yy.c
44452 lex.yy.c
$ gcc -o demo.exe lex.yy.c
$ printf '123 hello 456\n' | ./demo.exe
INT: 123
INT: 456
```

`hello` 被兜底规则 `.` 逐字符丢弃。这条兜底是必要的：没有它，Flex 的默认动作是把不匹配字符原样回显到标准输出，行为会与预期不符。

## 5.3 实验 2：多类数字字面量

表 5-1　`numbers.l` 规则设计（在给定框架上补写）

| 正规式 | 标记 | 设计理由 |
| --- | --- | --- |
| `0[xX][0-9a-fA-F]+` | HEX | 前缀大小写均可，后接至少一位十六进制数字 |
| `[0-9]+\.[0-9]+` | FLOAT | 小数点两侧强制至少一位数字，天然排除 `.5` 与 `1.` |
| `0[0-7]+` | OCT | 以 0 开头且后接至少一位 0–7，单独的 `0` 不会误判为八进制 |
| `0\|[1-9][0-9]*` | DEC | 不用 `[0-9]+`：那样 `075` 会被 DEC 与 OCT 等长匹配，类别只能由顺序决定 |

**终端实录 5-2** 主样例（与指南预期一致）

```
$ flex numbers.l && gcc -o numbers.exe lex.yy.c
$ printf '123 0x1F 075 3.14 0\n' | ./numbers.exe
DEC: 123
HEX: 0x1F
OCT: 075
FLOAT: 3.14
DEC: 0
```

**边界输入：先预测，再运行。两组输入均在纸上写下切分结果后才核对，11 个输入全部与预测一致：**

表 5-2 边界输入的预测与实际（第一组 / 第二组）

| 输入 | 预测 | 实际输出 | 结论 |
|---|---|---|---|
| `08` | DEC 0、DEC 8（8 不在 0–7） | DEC: 0 / DEC: 8 | 一致 |
| `0x` | DEC 0、UNHANDLED x | DEC: 0 / UNHANDLED: x | 一致 |
| `.5` | UNHANDLED .、DEC 5 | UNHANDLED: . / DEC: 5 | 一致 |
| `-12` | UNHANDLED -、DEC 12 | UNHANDLED: - / DEC: 12 | 一致 |
| `007` | OCT 007（3 > DEC 的 1） | OCT: 007 | 一致 |
| `0X1f` | HEX | HEX: 0X1f | 一致 |
| `1.` | DEC 1、UNHANDLED . | DEC: 1 / UNHANDLED: . | 一致 |
| `1.2.3` | FLOAT 1.2、UNHANDLED .、DEC 3 | FLOAT: 1.2 / UNHANDLED: . / DEC: 3 | 一致 |
| `0000` | OCT 0000 | OCT: 0000 | 一致 |
| `0x1G` | HEX 0x1、UNHANDLED G | HEX: 0x1 / UNHANDLED: G | 一致 |
| `12abc` | DEC 12、UNHANDLED a b c | DEC: 12 / UNHANDLED: a b c | 一致 |

**边界组真正揭示的是扫掠式识别的本质：局部匹配不等于整体合法——`08` 被切成两个 DEC，但 `08` 这个整体既非合法八进制也非合法十进制；`12abc` 同理。词法分析器只负责切分，合法性要靠后继阶段。负号必须由独立的运算符规则处理，并入数字规则会让 `a-12` 这类表达式切错；`.5`/`1.` 的不对称是"小数点两侧均有数字"这条规则设计的结果，不是缺陷。**

## 5.4 实验 3：关键字、标识符与整数

表 5-3 `mini_c.l` 规则组织（原始顺序）

| 顺序 | 正规式 | 标记 | 说明 |
|---|---|---|---|
| 1 | `int\|if\|while` | KW | 关键字 |
| 2 | `[A-Za-z_][A-Za-z_0-9]*` | ID | 字母或下划线开头 |
| 3 | `[0-9]+` | INT | 十进制整数 |
| 4 | `[ \t\r\n]+` `.` | — | 跳过空白；其余 `UNHANDLED` |

**终端实录 5-3** 主样例与最长匹配反例

```
$ printf 'int integer if ifx while while2 _x 42\n' | ./mini_c.exe
KW: int
ID: integer
KW: if
ID: ifx
KW: while
ID: while2
ID: _x
INT: 42

$ printf 'intx if_ while9 42x\n' | ./mini_c.exe
ID: intx
ID: if_
ID: while9
INT: 42
ID: x
```

`integer` 整体识别为 `ID` 而非 `KW: int` + `ID: eger`，因为标识符规则匹配 7 个字符，关键字只有 3 个——**最长匹配作用于所有规则**。`intx / if_ / while9` 三个反例进一步说明：只要标识符规则匹配更长，关键字就抢不走前缀。

表 5-4 交换关键字与标识符规则顺序后的对比

| 输入 | 关键字在前 | 标识符在前 | 原因 |
|---|---|---|---|
| `int` | KW: int | ID: int | 变了：等长并列，取靠前者 |
| `integer` | ID: integer | ID: integer | 不变：ID 匹配更长，与顺序无关 |
| `ifx` | ID: ifx | ID: ifx | 不变：3 > 2 |
| `while2` | ID: while2 | ID: while2 | 不变：同上 |

**终端实录 5-4** 交换顺序后 Flex 的静态诊断

```
$ win_flex mini_c_swapped.l
mini_c_swapped.l:7: warning, rule cannot be matched
$ printf 'int integer if ifx while while2 _x 42\n' | ./mini_c_swapped.exe
ID: int / ID: integer / ID: if / ID: ifx / ID: while / ID: while2 / ID: _x / INT: 42
```

`warning, rule cannot be matched` 说明交换后关键字规则已彻底不可达——任何能被它匹配的文本都能被前面的标识符规则匹配到同样长度。Flex 能在生成阶段静态发现这一点，本身就是"规则集合被编译成自动机"的证据。

## 5.5 最长匹配与等长规则优先级

Flex 的选择规则可精确写成两句：**最长匹配优先**（在所有能匹配当前位置的规则中选文本最长的，与书写顺序无关）；**等长时取靠前者**。自测反例是 `intx`：若实现采用"先试关键字、成功即返回"，会切成 `KW: int` + `ID: x`；实际输出 `ID: intx`，证明最长匹配在所有规则间统一比较。另一方向的反例是 `int` 本身：它无法用长度区分，只能靠顺序——这就是所有 C 风格词法分析器都把关键字规则写在标识符之前的原因。

## 5.6 从正规式到自动机

**终端实录 5-5** 生成文件中的自动机结构

```
$ grep -n "yy_accept\[" lex.yy.c | head -2
366:static const flex_int16_t yy_accept[20] =
696:            if ( yy_accept[yy_current_state] )
```

正规式先转成 NFA，多条规则合并后再确定化为 DFA，最后压成转移表与接受表；`yy_accept[state]` 给出"到达该状态时接受哪条规则"，第 696 行的判断是"若当前状态是接受态就记下来，以便继续试更长的匹配"。最长匹配在实现上就是不断前进，把最近一次遇到的接受态记下来，走到不能再走时回退到该位置。这也解释了 5.3 节的设计：`0|[1-9][0-9]*` 让自动机读完 `0` 之后对"下一位是数字"不再有接受路径，不会与 OCT 造出两个等长接受态。具体构造算法（Thompson 构造、子集化）在项目 2 中学习。

# 第 6 章　思政融合：国产编译器调研

## 6.1 调研对象

本次调研选择华为毕昇编译器（BiSheng Compiler）。按 openEuler 软件包仓库的官方描述，它是"华为编译器实验室针对鲲鹏等通用处理器架构场景，打造的一款高性能、高可信及易扩展的编译器工具链"[1]。选择它的理由很直接：它是国内少数已进入主流 Linux 发行版软件源、能被普通用户 `yum install` 下来直接使用的自研编译器，而非停留在发布会上的概念产品。

## 6.2 关键发现

支持语言与前端组成。毕昇不自己发明语言，而是复用 LLVM 生态：C/C++ 交由 Clang 完成词法、语法、语义分析与 LLVM IR 生成并兼任驱动，Fortran 交由 Flang（由 `flang1` 与 `flang2` 两个组件构成）[1]。这与第 2 章看到的 GCC 结构形成对照：GCC 的 `cc1` 是高度集成的单体程序，LLVM 系前端则按库拆分。

目标架构。官方文档把"架构"一栏写死为 AArch64，软件源路径为 `repo.oepkgs.net/bisheng/aarch64/`[1]——它的定位不是通用跨平台编译器，而是面向 ARM 服务器、特别是鲲鹏处理器的深度定制。鲲鹏社区的文章印证了这条路线：面向 x86 迁移到鲲鹏的场景，华为提供毕昇编译器、毕昇 JDK、GCC for openEuler 三件套，分别对应高性能计算、Java 业务与通用场景[2]。

工具链组成。编译 C/C++ 用 `clang` / `clang++`，Fortran 用 `flang`；链接默认调系统 `ld`，可用 `-fuse-ld=lld` 换成 LLVM 的 lld[1]——编译器与链接器解耦，与第 3.6 节 `-###` 看到的 `collect2 → ld` 是同一套分工。openEuler 已把它收进官方源，`yum install bisheng-compiler` 即装。

一项有来源支持的技术特征。官方表述为："除 LLVM 通用功能和优化外，毕昇编译器的工具链对中端及后端的关键技术点进行了深度优化，并集成 Auto-tuner 特性支持编译器自动调优"[1]。两个可核查点：优化发生在中端（IR 层）与后端（目标代码层），与第 4 章观察到的 `-O1` 常量折叠、`-O2` 死代码消除属同一层级；Auto-tuner 靠反复编译、运行基准并测量来自动搜索编译参数，本质上是把第 4 章"按场景选选项"那张表交给机器迭代。性能方面，鲲鹏社区口径为 SPEC CPU 2017 相比开源 GCC 9.3 提升约 25%[2]——这是厂商公布的测法，不宜直接外推到自己手上的程序。

## 6.3 我的思考

做完项目 1 再看毕昇，最直接的感受是"四阶段"是跨编译器的通用语言：`clang -E/-S/-c` 与 GCC 同名选项语义一致，因为二者都是"驱动 + 前端 + 优化器 + 后端 + 汇编器 + 链接器"的结构。第 3 章学到的观察方法——`-E` 看宏展开、`-S` 看指令、`nm` 看符号、`objdump -r` 看重定位——换任何一款类 LLVM 编译器都仍然有效。工具会换，方法论不会。

差异背后的原因更值得记。GCC 的 `cc1` 是单体可执行文件，Clang 是库化的：`libclang` 可被 IDE、静态分析器直接调用，这是 LLVM 生态在工具化方向扩张快的原因，也解释了国产编译器为何普遍选"基于 LLVM 深度定制"而非"从零写一个 GCC"——站在成熟的开源基础设施上做增量创新，是更

现实的技术路线。局限同样清楚：架构绑定 AArch64，x86 开发机用不了；依赖系统已有的 GCC（≥4.8.5）、glibc（≥2.17）、libstdc++（≥6）[1]，并非自包含工具链；Auto-tuner 靠反复编译测量收敛，更适合发布前的最终构建而非日常开发。

关于"一次可复现体验"，我设计了如下验证方案（本机为 x86_64 Windows，无法直接执行）：在 openEuler AArch64 上安装毕昇后，对同一份 `hello.c` 分别执行 `clang -E/-S/-c`，与本次 GCC 的三份产物并排比较——预期 `.i` 行标记格式、`.s` 伪指令（GCC 用 `.seh_*`）、`.o` 节名与符号修饰会有差异，但四个阶段的划分、`nm` 中 `T`/`U` 的含义、重定位项应当一致。若一致，即证明"四阶段"是工具链层面的客观结构，而非某一家编译器的实现细节。编译器为什么是基础软件，第 4 章也给了具象答案：同一段循环 `-O0` 下 11 ms、`-O2` 下被彻底消除——编译器不动一行源码，就决定了程序的性能上限；把尺度放大到一座超算集群，这个差距就是电费与机时。

## 6.4 参考文献


[1] openEuler OEPKGS. huawei/bisheng（毕昇编译器仓库说明）[EB/OL]. https://docs.oepkgs.net/zh-CN/Repos/huawei-bisheng/，访问日期 2026-10-25.

[2] 鲲鹏社区. GCC for openEuler 与鲲鹏编译工具链介绍[EB/OL]. https://www.hikunpeng.com.cn/forum/thread-0225203517001409120-1-1.html，访问日期 2026-10-25.

[3] LLVM Project. Clang Compiler User's Manual[EB/OL]. https://clang.llvm.org/docs/UsersManual.html，访问日期 2026-10-25.

[4] GCC Team. Overall Options (Using the GNU Compiler Collection)[EB/OL]. https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html，访问日期 2026-10-25.

# 第 7 章　综合反思

## 7.1 最有趣的环节

`-O1` 已经把 `sum` 折叠成常量，却留下了执行 1000 万次的空计数循环（4.1 节），于是"算术都不做了，计时还有 3 毫秒"。它推翻了"优化等级是单调滑杆"的直觉——真实情况是一组彼此独立的变换，各自有不同触发条件：常量折叠 `-O1` 就做，死循环消除要等 `-O2`。`-O1` 的"半优化状态"最需要警惕：既丢了可调试性，又没拿到性能。

## 7.2 最大收获

"编译"从一个词变成了一条流水线：五样文件、四个阶段、符号解析与重定位两个机制。三条最实用的经验——出错先判断错在哪个阶段（语法错带行列号发生在编译期，`undefined reference` 报的是目标文件名，发生在链接期）；`-###` 是理解工具链最快的入口；能交给正规式表达的歧义，不要留给规则顺序兜底（DEC 写成 `0|[1-9][0-9]*` 而非 `[0-9]+`，把"`075` 算 DEC 还是 OCT"在源头消除）。

## 7.3 还存在的疑问

一是为什么 `-O2` 的可执行文件反而比 `-O0` 大 36 字节——代码段明明更小，推测与 PE 节对齐和导入表布局有关，尚未逐节比对；二是 `clock()` 在 Windows 上的实际分辨率与时钟源，`-O0` 九次采样从 0.008 s 波动到 0.017 s，跨度接近一倍，需要确认是否受 CPU 频率调节影响；三是 Flex 生成文件里 `yy_accept[20]` 只有 20 项而规则状态显然更多，说明状态经过重映射与压缩，等价类合并的细节留待项目 2；四是第 6 章设计的毕昇验证方案尚缺 AArch64 实机执行。方法上的一点体会：凡是"预期"与"实测"不一致的地方都原样记录（`PE32+` 与 `ELF`、`WinMain` 与 `main`），这些差异恰好说明四阶段是抽象结构，格式与命名只是实现细节。

# 附录 A　提交文件清单与自行完成部分

表 A-1　提交的源文件（均随 `src/` 目录提交）

| 序号 | 文件 | 对应任务 | 说明 |
|---|---|---|---|
| 1-5 | `hello.c` `bug.c` `bug_fixed.c` `add.c` `main.c` | 1.1 / 1.2 | 编译运行、语法诊断（`bug.c` 保留不修复）、多文件链接 |
| 6 | `loop.c` | 1.3-1 | 循环计时程序，`N = 10 000 000` |
| 7-8 | `warn.c` / `warn_fixed.c` | 1.3-2 | 5 类缺陷诊断样本（仅编译）及修复版 |
| 9 | `test.c` | 1.3-3/4 | `-g` 与 `-DDEBUG` 对比用 |
| 10 | `demo.l` | 1.4-1 | 最简整数识别器（工具示例） |
| 11 | `numbers.l` | 1.4-2 | 自行完成四类数字字面量规则 |
| 12 | `mini_c.l` | 1.4-3 | 自行完成关键字/标识符/整数规则 |

## A.1 `numbers.l`（自行完成部分）

```
%option noyywrap
%{
#include <stdio.h>
%}
%%
0[xX][0-9a-fA-F]+   { printf("HEX: %s\n", yytext); }
[0-9]+\.[0-9]+      { printf("FLOAT: %s\n", yytext); }
0[0-7]+             { printf("OCT: %s\n", yytext); }
0|[1-9][0-9]*       { printf("DEC: %s\n", yytext); }
[ \t\r\n]+          { /* skip whitespace */ }
.                   { printf("UNHANDLED: %s\n", yytext); }
%%
int main(void) { yylex(); return 0; }
```

## A.2 `mini_c.l`（自行完成部分）

```
%option noyywrap
%{
#include <stdio.h>
%}
%%
int|if|while           { printf("KW: %s\n", yytext); }
[A-Za-z_][A-Za-z_0-9]* { printf("ID: %s\n", yytext); }
[0-9]+                 { printf("INT: %s\n", yytext); }
[ \t\r\n]+             { /* skip whitespace */ }
.                      { printf("UNHANDLED: %s\n", yytext); }
%%
int main(void) { yylex(); return 0; }
```

— 报告完 —